



Session 04:

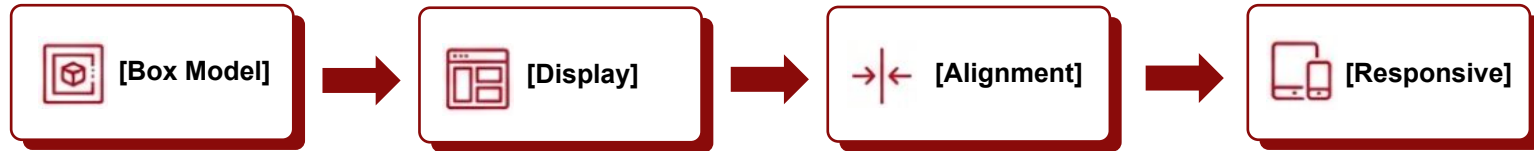
CSS Box Model, Display & Media Queries



1. **Nắm vững cấu trúc Box Model cốt lõi**
2. **Phân biệt và ứng dụng các thuộc tính Display**
3. **Làm chủ các kỹ thuật Alignment (căn chỉnh)**
4. **Viết Media Queries cho Responsive Web**

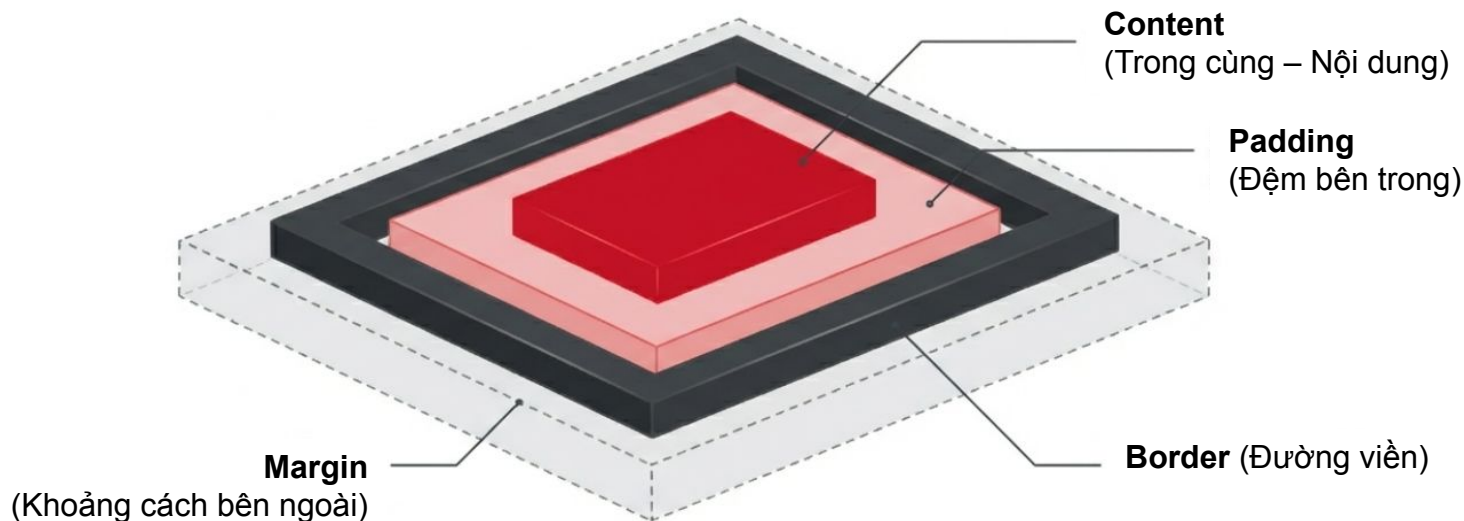
Mọi thứ trên Web đều là các “Khối hộp”

Cách chúng ta quản lý kích thước, luồng chảy (flow) và phản ứng của các khối hộp này sẽ tạo nên toàn bộ Bố cục (Layout)



Mô hình hộp (Box Model) là gì?

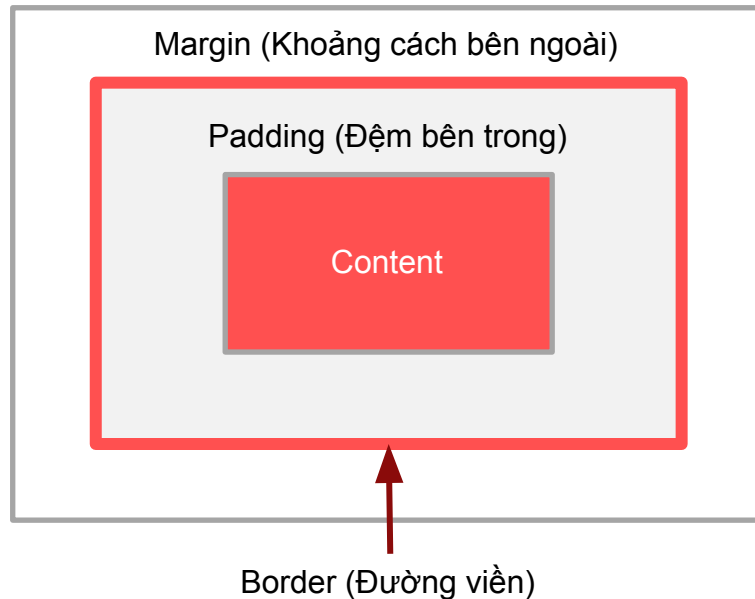
Trình duyệt nhìn mọi phần tử HTML dưới dạng một hình chữ nhật. Box Model quy định cách tính toán không gian và kích thước của hình chữ nhật đó.



Chi tiết 4 thành phần Box Model



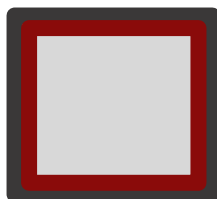
```
.box {  
  padding: 10px;  
  border: 2px solid red;  
  margin: 20px;  
}
```



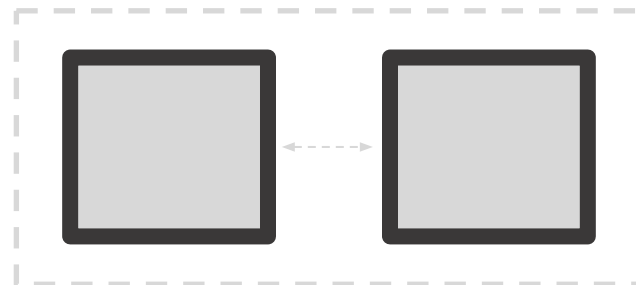
Phân biệt Padding và Margin

Tiêu chí	Padding	Margin
Vị trí	Bên trong đường viền (Border)	Bên ngoài đường viền
Tác dụng	Đẩy nội dung xa khỏi viền	Đẩy các phần tử khác ra xa
Màu nền (Background)	Hiển thị màu nền của phần tử	Hoàn toàn trong suốt

Padding



Margin



“Cú lừa” của kích thước mặc định

Mặc định (content-box), width chỉ tính phần Content. Padding và Border sẽ cộng dồn và làm phình to hộp

$$\text{width: } 100\text{px} + \text{padding: } 20\text{px (x2)} + \text{border: } 2\text{px (x2)} = \underline{144\text{px}}$$

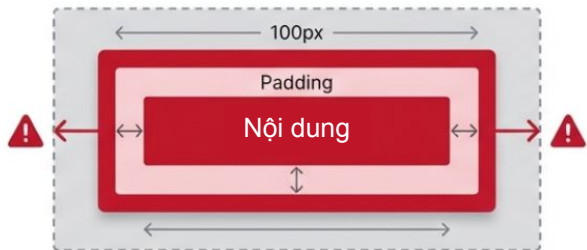
Thực tế



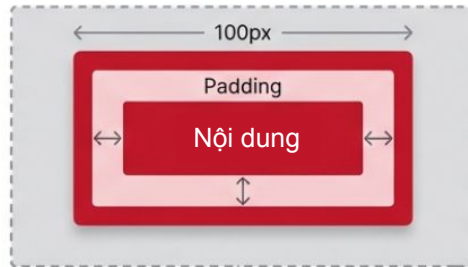
Giải pháp tối thượng: box-sizing

box-sizing: border-box ép trình duyệt bao gồm cả padding và border vào trong width/height đã khai báo

content-box



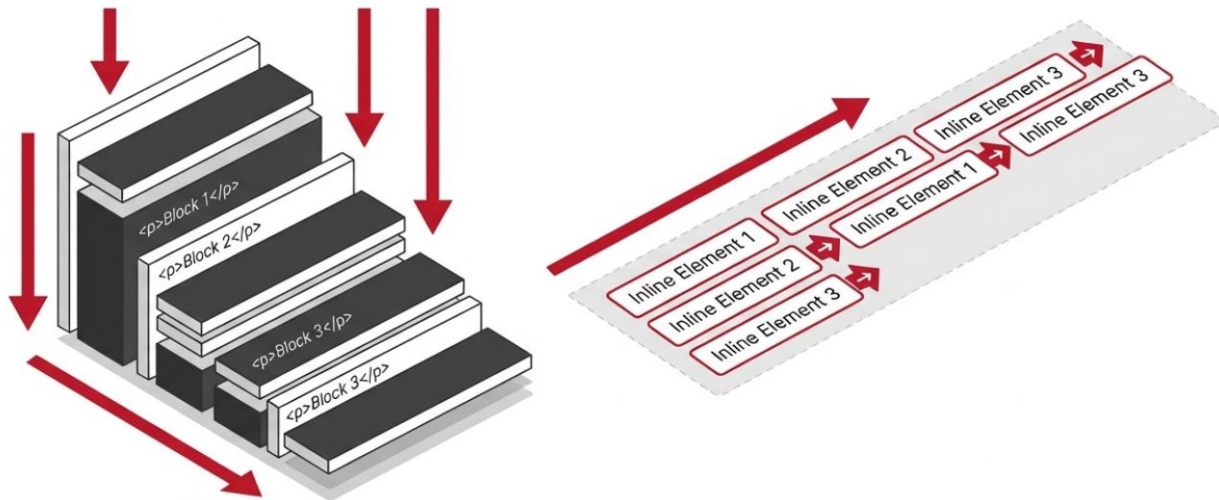
border-box



```
* {  
  box-sizing: border-box;  
}
```

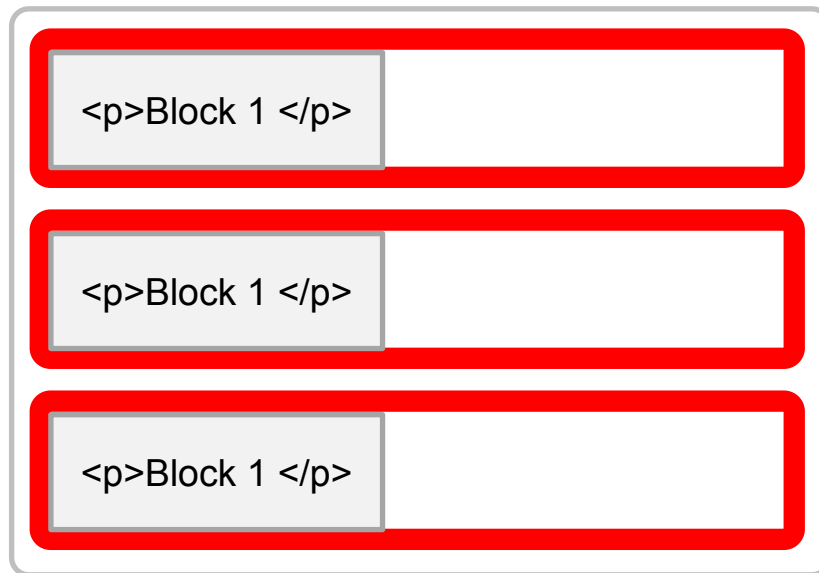

Bản chất luồng hiển thị (Normal Flow)

Trình duyệt mặc định đọc HTML và sắp xếp các phần tử từ trên xuống dưới, từ trái qua phải. Thuộc tính display quyết định cách phần tử tham gia vào luồng này.



Thuộc tính Display: Block

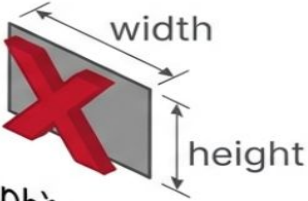
- ▶ Chiếm 100% chiều rộng có sẵn của thẻ cha.
- ▶ Luôn bắt đầu trên một dòng mới.
- ▶ Có thể thiết lập width và height.
- ▶ Đại diện tiêu biểu: <div>, <p>, <h1>.



Thuộc tính Display: Inline

- ▶ Chỉ chiếm không gian vừa đủ chứa nội dung.
- ▶ Nằm san sát nhau trên cùng một dòng.
- ▶ **KHÔNG THỂ** thiết lập width và height.
- ▶ Đại diện tiêu biểu: `<span>`, `<a>`, `<strong>`

Trong một đoạn văn, các phần tử inline như `<span>`, `<a>` liên kết, và `<strong>` đậm `EI>` sẽ nằm cùng một dòng với văn bản xung quanh.

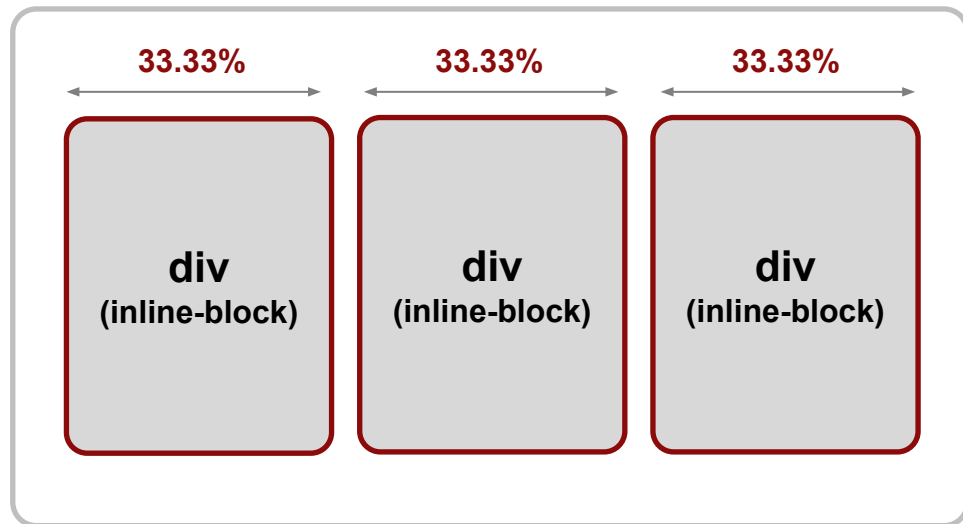


Sự kết hợp hoàn hảo: Inline-Block

Nằm trên cùng 1 dòng (như inline), nhưng **CÓ THỂ** thiết lập kích thước (như block). Giải pháp chia cột truyền thống cực mạnh!

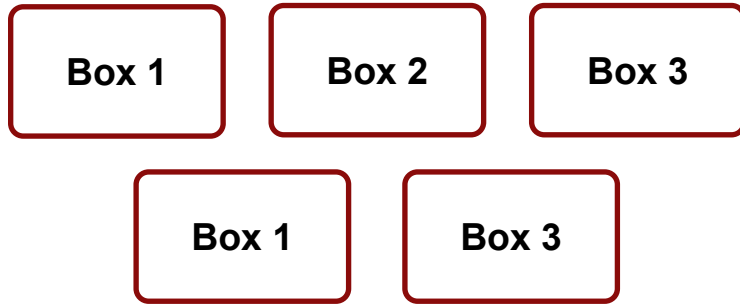


```
.column {  
  display: inline-block;  
  width: 33.33%;  
}
```



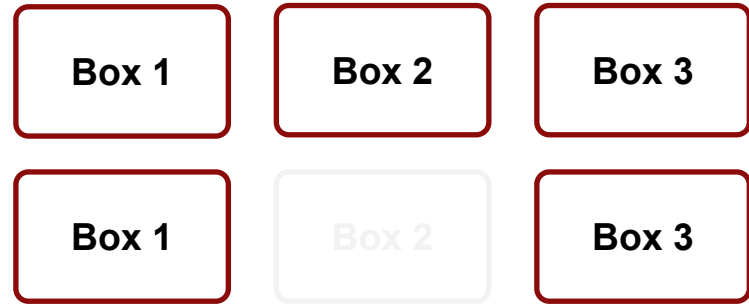
Ẩn phần tử: display vs visibility

Trường hợp 1: **display: none**



Xóa hoàn toàn khỏi layout

Trường hợp 2: **visibility: hidden**



Tàng hình, vẫn giữ nguyên không gian chiếm chỗ

Căn giữa nội dung văn bản & Inline

Sử dụng **text-align: center** trên phần tử cha (Block) để căn giữa tất cả nội dung con bên trong nó (Text, Inline, Inline-block).

```
1 <div class="container">
2   <h1>Tiêu đề</h1>
3   <button>Click Me</button>
4 </div>
5
6 .container {
7   text-align: center;
8 }
```

Tiêu đề

Click me

Căn giữa phần tử Block

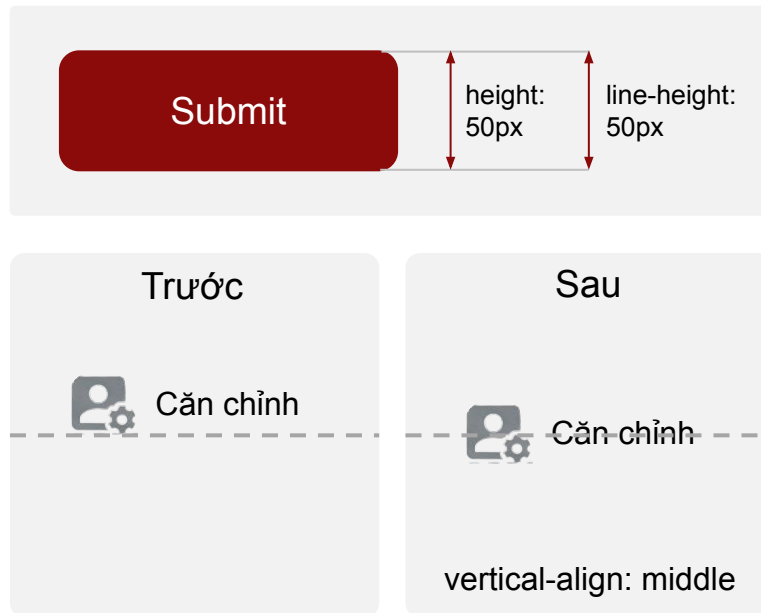
Để căn chính giữa một khối Block, phải khai báo **width** cụ thể và dùng **margin: 0 auto;**

```
.box {  
  width: 500px;  
  margin: 0 auto;  
}
```



Căn dọc cơ bản (Vertical Alignment)

- ▶ **Căn giữa chữ 1 dòng:** Đặt line-height bằng chính height của thẻ cha
- ▶ **Căn chỉnh icon/ảnh với chữ:** Sử dụng **vertical-align: middle**.



Kỹ thuật Float (Khái niệm kế thừa)

Ban đầu sinh ra để làm nội dung chữ bao quanh hình ảnh (Text wrap). Từng được tận dụng để chia cột trước thời đại Flex/Grid



Thiết kế đáp ứng (Responsive Web Design)

Trang web tự động thích nghi với mọi kích thước màn hình (PC, Tablet, Mobile) mà không làm vỡ giao diện. Thẻ meta viewport là bắt buộc trong HTML



Cú pháp & Điểm ngắt (Breakpoints)

Định nghĩa Media Query

```
@media (max-width: 768px) {
  /* CSS chạy khi màn hình <= 768px */
  .column {
    width: 100%;
    display: block;
  }
}
```

Điểm ngắt (Breakpoint)



Mobile-First: Code cho Mobile trước,
dùng min-width nâng cấp lên PC



Desktop-First: Code cho PC trước,
dùng max-width thu gọn cho Mobile

So sánh Mobile-First và Desktop-First

	Mobile-First (Ưu tiên di động) – Xu hướng hiện nay	Desktop-First (Ưu tiên máy tính)
Đặc điểm	Thiết kế và code cho thiết bị di động trước. Sử dụng `min-width` media queries để nâng cấp dần lên các màn hình lớn hơn.	Thiết kế và code cho PC trước. Sử dụng `max-width` media queries để thu gọn và điều chỉnh cho các màn hình nhỏ hơn.
CSS Media Query	<pre>@media (min-width: 768px) { ... }</pre>	<pre>@media (max-width: 1024px) { ... }</pre>
Quy trình phát triển		

- ❑ Hiểu và nắm vững 4 thành phần cấu tạo nên mô hình CSS Box Model: Content, Padding, Border, Margin
- ❑ Phân biệt rõ sự khác nhau giữa Padding (đẩy nội dung vào trong) và Margin (đẩy các phần tử khác ra xa)
- ❑ Nên sử dụng box-sizing: border-box để ép trình duyệt bao gồm cả padding và border vào trong kích thước width/height đã khai báo, tránh làm vỡ layout
- ❑ Phân biệt và ứng dụng đúng lúc 3 thuộc tính Display cốt lõi: block (chiếm 100% chiều rộng), inline (chỉ chiếm không gian chứa nội dung) và inline-block (kết hợp cả hai)
- ❑ Nắm vững sự khác biệt khi ẩn phần tử: display: none (xóa hoàn toàn khỏi layout) so với visibility: hidden (tàng hình nhưng vẫn giữ không gian chiếm chỗ)
- ❑ Làm chủ các kỹ thuật căn chỉnh (Alignment): Dùng margin: 0 auto để căn giữa khối Block, và text-align: center trên phần tử cha để căn giữa nội dung inline/inline-block
- ❑ Xử lý căn dọc cơ bản: Đặt line-height bằng với height để căn chữ 1 dòng, hoặc dùng vertical-align: middle để căn chỉnh ảnh/icon với chữ
- ❑ Hiểu tư duy thiết kế đáp ứng (Responsive Web Design) và viết thành thạo Media Queries để giao diện thích nghi với các kích thước màn hình
- ❑ Phân biệt và áp dụng đúng luồng điểm ngắt (Breakpoints) cho hai chiến lược: Mobile-First (dùng min-width) và Desktop-First (dùng max-width)



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN